Towards Practical Formal Verification of Smart Contracts

– Technical Report –

DRAFT – Implementation Only View

David Dill Wolfgang Grieskamp Junkil Park
Shaz Qadeer Meng Xu Emma Zhong

August 27, 2021

Contents

3 Move Prover Implementation **2**
3.1 Basic Architecture . . . 2
3.2 Reference Elimination . . . 3
3.2.1 Immutable References . . . 3
3.2.2 Mutable References . . . 4
3.3 Function Condition Injection . . . 6
3.3.1 Modular Veri cation . . . 7
3.3.2 Pre- and Post conditions . . . 7
3.3.3 Modi es . . . 8
3.3.4 Data Invariants . . . 9
3.4 Global Invariant Injection . . . 10
3.4.1 Basic Translation . . . 11
3.4.2 Genericity . . . 12
3.4.3 Modularity . . . 12
3.4.4 Suspending Invariants . . . 13
3.4.5 Invariant Consistency . . . 15
3.5 Monomorphization . . . 15
3.5.1 Basic Monomorphization . . . 15
3.5.2 Type Dependent Code . . . 16
3.6 Translation to Boogie and Z3 . . . 17
3.6.1 Vectors and Extensionality . . . 17
3.6.2 Encoding . . . 17
3.6.3 Butter ies . . . 17

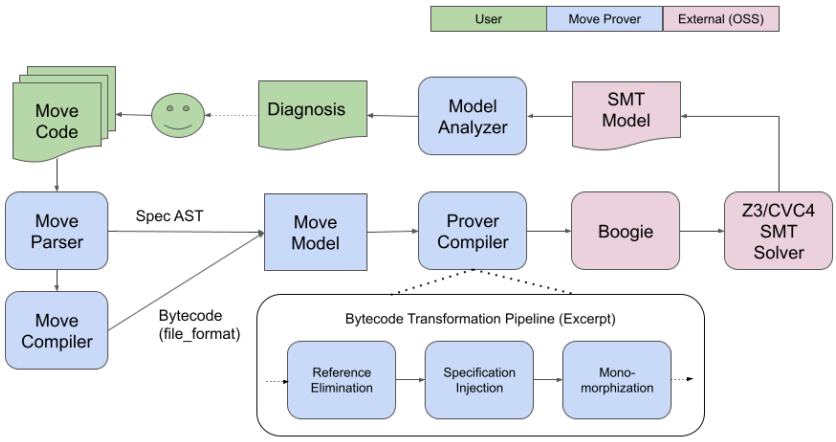

Figure 1: Move Prover Architecture

3 Move Prover Implementation

In this section, an overview of the Move Prover implementation will be provided. The formal content of the discussion is kept lightweight; a formalization of some aspects is given in appendices.

3.1 Basic Architecture

The architecture of the Move Prover is illustrated in Fig. 1. Move code (consisting of Move programs and speci cations) is given as input to the Move tool chain, which produces two artifacts: the abstract syntax tree (AST) of the speci cations in the code, as well as the translated Move bytecode for the program part. It is essential that the Prover interprets the Move program on bytecode level, not on the intermediate AST: this way we verify the \source of truth" which is also executed in the Move VM. Only the speci cation parts are passed on as AST. The *Move Model* is a component which merges both bytecode and speci cations, as well as other metdata from the original code, into a unique object model which is input to the remaining tool chain.

The next phase is the actual *Prover Compiler*, which is implemented as a pipeline of bytecode transformations. Only an excerpt of the most important transformations is shown (Reference Elimination, Speci cation Injection, and Monomorphization). These transformations will be conceptually described in more detail in subsequent sections. While they happen in reality on an extended version of the Move bytecode, we will illustrate them on a higher level of abstraction, as Move source level transformations.

The transformed bytecode is next compiled into the Boogie intermediate veri cation language [3]. Boogie supports an imperative programming model which is well suited for the encoding of the transformed Move code. Boogie in

turn can translate to multiple SMT solver backends, namely Z3 [5] and CVC5 [4]; the default choice for the Move prover is currently Z3.

When the SMT solver produces a `sat` or `unknown` result (of the negation of the veri cation condition Boogie generates), it produces a model witness. The Move Prover undertakes some e ort to translate this model back into diagnosis which a user can associate it with the original Move code (as has been illustrated in Sec. **??**.) For example, execution traces leading to the veri cation failure are shown, with assignments to variables used in this trace, extracted from the model. Also the Move Model will be consulted to retrieve the original source information and display it with the diagnosis.

Subsequently, we will focus on the major bytecode transformations as well as the encoding and translation to Boogie.

3.2 Reference Elimination

The Move language supports references to data stored in global memory and on the stack. Those references can point to interior parts of the data. The reference system is based on *borrow semantics* [2] as it is also found in the Rust programming language. One can create (immutable) references `&x` and mutable references `&mut x`, and derive new references by eld selection (`&mut x.f` and `&x.f`). The borrow semantics of Move provides the following guarantees (ensured by the borrow checker [1]):

- For any given location in global memory or on the stack, there can be either exactly one mutable reference, or n immutable references. Hereby, it does not matter to what interior part of the data is referred to.
- Dangling references to locations on the stack cannot exist; that is, the lifetime of references to data on the stack is restricted to the lifetime of the stack location.

These properties enable us to *effectively eliminate* references from the Move program, reducing the veri cation complexity signi cantly, as we do not need to reason about sharing. It comes as no surprise that the same discipline of borrowing which makes Move (and Rust) programs safer by design also makes veri cation simpler.

3.2.1 Immutable References

Since during the existance of an immutable reference no mutation on the referenced data can occur, we can simply replace references by the referred value.

An example of the applied transformation is shown below. We remove the reference type constructor and all reference-taking operations from the code:

```
fun select_f(s: &S): &T { &s.f } ⇝ fun select_f(s: S): T { s.f }
```

Notice that at Move execution time, immutable references serve performance objectives (avoid copies); however, the symbolic reasoning engines we use have

a di erent representation of values, in which structure sharing is common and copying is cheap.

3.2.2 Mutable References

Each mutation of a location `l` starts with an initial borrow for the whole data stored in this location (in Move, **borrow_global_mut**`<T>(addr)` for global memory, and **&mut** `x` for a local on the stack). Let's call the reference resulting from such a borrow `r`. As long as this reference is alive, Move code can either update its value (`*r = v`), or replace it with a sub-reference (`r' =` **&mut** `r.f`). The mutation ends when `r` (or the derived `r'`) go out of scope. Because of the guarantees of the borrow semantics, during the mutation of the data in `l` no other reference can exist into data in `l`.

The fact that **&mut** has exclusive access to the whole value in a location allows to reduce mutable references to a *read-update-write* cycle. One can create a copy of the data in `l` and single-thread it to a sequence of mutation steps which are represented as purely functional data updates. Once the last reference for the data in `l` goes out of scope, the updated value is written back to `l`. This e ectively turns an imperative program with references into an imperative program which only has state updates on global memory or variables on the stack, a class of programs which is known to have a signi cant simpler semantics. We illustrate the basics of this approach by an example:

```
fun increment(x: &mut u64) { *x = *x + 1 }
fun increment_field(s: &mut S) { increment(&mut s.f) }
fun caller(): S { let s = S{f:0}; update(&mut s); s }
⇝
fun increment(x: u64): u64 { x + 1 }
fun increment_field(s: S): S { s[f = increment(s.f)] }
fun caller(): S { let s = S{f:0}; s = update(s); s }
```

While the setup in this example covers a majority of the uses cases in every day Move code, there are more complex ones to consider, namely that the value of a reference depends on runtime decisions:

```
let r = if (p) &mut s1 else &mut s2;
increment_field(r);
```

Additional runtime information is required to deal with such cases. At the execution point a reference goes out of scope, we need to know from which location it was derived from, so we can write back the updated value correctly. Fig. 3.2.2 illustrates the approach for doing this. A new Move prover internal type `Mut<T>` is introduced which carries the location from which `T` was derived together with the value. It supports the following operations:

- `Mvp::mklocal(value, LOCAL_ID)` creates a new mutation value for a local with the given local id. Local ids are transformation generated constants kept opaque here.

- Similarily, `Mvp::mkglobal(value, TYPE_ID, addr)` creates a new mutation for a global with given type and address. Notice that in the current Move type system, we would not need to represent the address, since there can be only one mutable reference into the entire type (via the acquires mechanism). However, we keep this more general here, as the Move type system might change.
- With `r' = Mvp::field(r, FIELD_ID)` a mutation value for a subreference is created for the identi ed eld.
- The value of a mutation is replaced with `r' = Mvp::set(r, v)` and retrieved with `v = Mvp::get(r)`.
- With the predicate `Mvp::is_local(r, LOCAL_ID)` one can test whether `r` was derived from the given local, and with `Mvp::is_global(r, TYPE_ID, addr)` whether it was derived from the speci ed global. The predicate `Mvp::is_field(r, FIELD_ID)` tests whether it is derived from the given eld.

Implementation The Move Prover has a partial implementation of the illustrated transformation. The completeness of this implementation has not yet been formally investigated, but we believe that it covers all of Move, with the language's simpli cation that we do not need to distinguish addresses in global memory locations.[1] (See discussion of `Mvp::mkglobal` above.) The transformation also relies on that in Move there are no recursive data types, so eld selection paths are statically known. While those things can be potentially generalized, we have not yet investigated this direction.

The transformation constructs a *borrow graph* from the program via a data ow analysis. The borrow graph tracks both when references are released as well as how they relate to each other: e.g. `r' = &mut r.f` creates a edge from `r` to `r'` labelled with `f`, and `r' = &mut r.g` creates another also starting from `r`. For the matter of this problem, a reference is not released until a direct or indirect borrow on it goes out of scope; notice that its lifetimes in terms of borrowing is larger than the scope of its usage. The borrow analysis is *inter-procedural* requiring computed summaries for the borrow graph of called functions.

The resulting borrow graph is then used to guide the transformation, inserting the operations of the `Mut<T>` type as illustrated in Fig 3.2.2. Speci cally, when the borrow on a reference ends, the associated mutation value must be written back to its parent mutation or the original location (e.g. line 29 in Fig. 3.2.2). The presence of multiple possible origins leads to case distinctions via `Mvp::is_X` predicates; however, these cases are rare in actual Move programs.

Performance **TODO(wrwg)**: We may want to identify some historical benchmarks before memory model.

[1] **TODO(wrwg)**: Need to investigate loops!

Figure 2: Elimination of Mutable References

```
fun increment(x: &mut u64) { *x = *x + 1 }
fun increment_field(s: &mut S) {
  let r = if (s.f > 0) &mut s.f else &mut s.g;
  increment(r)
}
fun caller(p: bool): (S, S) {
  let s1 = S{f:0, g:0}; let s2 = S{f:1, g:1};
  let r = if (p) &mut s1 else &mut s2;
  increment_field(r);
  (s1, s2)
}
⇝
fun increment(x: Mut<u64>): Mut<u64> { Mvp::set(x, Mvp::get(x) + 1) }
fun increment_field(s: Mut<S>): Mut<S> {
  let r = if (s.f > 0) Mvp::field(s.f, S_F) else Mvp::field(s.g, S_G);
  r = increment(r);
  if (Mvp::is_field(r, S_F))
    s = Mvp::set(s, Mvp::get(s)[f = Mvp::get(r)]);
  if (Mvp::is_field(r, S_G))
    s = Mvp::set(s, Mvp::get(s)[g = Mvp::get(r)]);
  s
}
fun caller(p: bool): S {
  let s1 = S{f:0, g:0}; let s2 = S{f:1, g:1};
  let r = if (p) Mvp::mklocal(s1, CALLER_s1)
          else Mvp::mklocal(s2, CALLER_s2);
  r = increment_field(r);
  if (Mvp::is_local(r, CALLER_s1))
    s1 = Mvp::get(r);
  if (Mvp::is_local(r, CALLER_s2))
    s2 = Mvp::get(r);
  (s1, s2)
}
```

3.3 Function Condition Injection

During speci cation injection, move speci cations are reduced to basic assume/assert statements added to the Move code. Those statements represent instructions to the solver backend about what propositions can be assumed and which need to be asserted (veri ed) at a given program point. In this section, we cover how *function specification conditions* are injected.

3.3.1 Modular Verification

Modular veri cation applies to all types of injections, and its principles are therefore described rst. When the Move prover is run, it takes as input a set of Move modules which is closed under the transitive dependency relation (module imports). However, only a subset of those modules are *verification target* (typically just one module). It is assumed that the tool environment ensures that modules in the dependency relation which are not target of veri cation have already successfully veri ed. This is possible since Move has an acyclic import relation.

From the set of target modules, the set of *target functions* is derived. This set might be enriched by additional functions which need veri cation because of global invariants, as discussed in Sec. 3.4. The resulting set of target functions will then be veri ed one-by-one, assuming that any called functions have successfully veri ed. If a called function is among the target functions, it might in fact not verify; however, in this case a veri cation error will be reported at the called function, and the veri cation result at the caller side can be ignored.

3.3.2 Pre- and Post conditions

The injection of basic function speci cations is illustrated in Fig. 3. An extension of the Move source language is used to specify abort behavior. With **fun** `f() { .. }` **onabort** `{ conditions }` a Move function is de ned where `conditions` are assume or assert statements that are evaluated at every program point the function aborts (either implicitly or with an **abort** statement). This construct simpli es the presentation and corresponds to a per-function abort block on bytecode level which is target of branching.

An aborts condition is translated into two di erent asserts: one where the function aborts and the condition must hold (line 21), and one where it returns and the condition must *not* hold (line 17). If there are multiple **aborts_if**, they are or-ed. If there is no aborts condition, no asserts are generated. This means that once a user speci es aborts conditions, they must completely cover the abort behavior of the code. (The prover also provides an option to relax this behavior, where aborts conditions can be partial and are only enforced on function return.)

For a function call site we distinguish two variants: the call is *inlined* (line 25) or it is *opaque* (line 27). In both cases, it is assumed that the called function is veri ed (see Modular Veri cation, Sec. 3.3.1). For inlined calls, the function de nition, with all injected assumptions and assertions turned into assumptions (as those are considered proven) is substituted. For opaque functions the speci cation conditions are inserted as assumptions. Methodologically, opaque functions need precise speci cations relative to a particular objective, where as in the case of inlined functions the code is still the source of truth and speci - cations can be partial or omitted. However, inlining does not scale arbitrarily, and can be only used for small function systems.

Notice we have not discussed the way how to deal with relat-

Figure 3: Requires, Ensures, and AbortsIf Injection

```
fun f(x: u64, y: u64): u64 { x + y }
spec f {
  requires x < y;
  aborts_if x + y > MAX_U64;
  ensures result == x + y;
}
fun g(x: u64): u64 { f(x, x + 1) }
spec g {
  ensures result > x;
}
⇝
fun f(x: u64, y: u64): u64 {
  spec assume x < y;
  let result = x + y;
  spec assert result == x + y;     // ensures of of
  spec assert                      // negated abort_if of f
    !(x + y > MAX_U64);
  result
} onabort {
  spec assert                      // abort_if of f
    x + y > MAX_U64;
}
fun g(x: u64): u64 {
  spec assert x < x + 1;           // requires of f
if inlined
  let result = inline f(x, x + 1);
elif opaque
  if (x + x + 1 > MAX_U64) abort;  // aborts_if of f
  spec assume result == x + x + 1; // ensures of f
endif
  spec assert result > x;          // ensures of g
  result
}
```

ing pre and post states yet, which requires taking snapshots of state (e.g. **ensures** `x == old(x) + 1`); the example in Fig. 3 does not need it. Snapshots of state will be discussed for global update invariants in Sec. 3.4.

3.3.3 Modifies

The **modifies** condition speci es that a function only changes speci c memory. It comes in the form **modifies global**`<T>(addr)`, and its injection is illustrated in Fig. 4.

A type check is used to ensure that if a function has one or more **modifies** conditions all called functions which are *opaque* have a matching modi es dec-

Figure 4: Modi es Injection

```
fun f(addr: address) { move_to<T>(addr, T{}) }
spec f {
  pragma opaque;
  ensures exists<T>(addr);
  modifies global<T>(addr);
}
fun g() { f(0x1) }
spec g {
  modifies global<T>(0x1); modifies global<T>(0x2);
}
⇝
fun f(addr: address) {
  let can_modify_T = {addr};      // modifies of f
  spec assert addr in can_modify; // permission check
  move_to<T>(addr, T{});
}
fun g() {
  let can_modify_T = {0x1, 0x2};  // modifies of g
  spec assert {0x1} <= can_modify_T; // permission check
  spec havoc global<T>(0x1);      // havoc modified memory
  spec assume exists<T>(0x1);     // ensures of f
}
```

laration. This is important so we can relate the callees memory modi cations to that what is allowed at caller side.

At veri cation time, when an operation is performed which modi es memory, an assertion is emitted that modi cation is allowed (e.g. line 14). The permitted addresses derived from the modi es clause are stored in a set `can_modify_T` generated by the transformation. Instructions which modify memory are either primitives (like **move_to** in the example) or function calls. If the function call is inlined, modi es injection proceeds (conceptually) with the inlined body. For opaque function calls, the static analysis has ensured that the target has a modi es clause. This clause is used to derive the modi ed memory, which must be a subset of the modi ed memory of the caller (line 19).

For opaque calls, we also need to *havoc* the memory they modify (line 20), by which is meant assigning an unconstrained value to it. If present, **ensures** from the called function, injected as subsequent assumptions, are further constraining the modi ed memory.

3.3.4 Data Invariants

A data invariant speci es a constraint over a struct value. The value is guaranteed to satisfy this constraint at any time. Thus, when a value is constructed, the data invariant needs to be veri ed, and when it is consumed, it can be

Figure 5: Data Invariant Injection

```
struct S { a: u64, b: u64 }
spec S { invariant a < b }
fun f(s: S): S {
  let r = &mut s;
  r.a = r.a + 1;
  r.b = r.b + 1;
  s
}
⇝
fun f(s: S): S {
  spec assume s.a < s.b;      // assume invariant for s
  let r = Mvp::local(s, F_s); // begin mutation of s
  r = Mvp::set(r, Mvp::get(r)[a = Mvp::get(r).a + 1]);
  r = Mvp::set(r, Mvp::get(r)[b = Mvp::get(r).b + 1]);
  spec assert                 // invariant enforced
    Mvp::get(r).a < Mvp::get(r).b;
  s = Mvp::get(r);            // write back to s
  s
}
```

assumed to hold.

In Move's reference semantics, construction of struct values is often done via a sequence of mutations via mutable references. It is desirable that *during* such mutations, assertion of the data invariant is suspended. This allows to state invariants which reference multiple elds, where the elds are updated step-by-step. Move's borrow semantics and concept of mutations provides a natural way how to defer invariant evaluation: at the point a mutable reference is released, mutation ends, and the data invariant can be enforced. In other speci cation formalisms, we would need a special language construct for invariant suspension. Fig. 5 gives an example, and shows how data invariants are reduced to assert/assume statements.

Implementation The implementation hooks into the reference elimination (Sec. 3.2). As part of this the lifetime of references is computed. Whenever a reference is released and the mutated value is written back, we also enforce the data invariant. In addition, the data invariant is enforced when a struct value is directly constructed.

3.4 Global Invariant Injection

Global invariants appear on Move module level and constraint the content of the memory. While the basic injection of global invariants is relative simple, they cause signi cant complexity with features like modular veri cation, suspension,

and generics. We first discuss the basic model, then extend it step wise.

3.4.1 Basic Translation

Figure 6: Basic Global Invariant Injection

```
fun f(a: address) {
  let r = borrow_global_mut<S>(a);
  r.value = r.value + 1
}
invariant [I1] forall a: address: global<S>(a).value > 0;
invariant [I2] update forall a: address:
    global<S>(a).value > old(global<S>(a).value);
⇝
fun f(a: address) {
  spec assume I1;
  Mvp::snapshot_state(I2_BEFORE);
  r = <increment mutation>;
  spec assert I1;
  spec assert I2[old = I2_BEFORE];
}
```

Fig. 6 contains an example for the supported invariant types and their injection into code. The first invariant, `I1`, is a regular state invariant. It is assumed on function entry, and asserted after the state update. The second, `I2`, is a state update invariant, which relates pre and post states. For this a state snapshot is stored under some label `I2_BEFORE`, which is then used in an assertion.

Global invariant injection is optimized by knowledge of the prover, obtained by static analysis, about (transitively) accessed memory. For opaque functions (including also builtin functions) this information is obtained via the modifies clause. For other functions it is determined from the code. Assuming that the prover has precise knowledge (up to symbolic address representation) of memory usage, it can determine which invariants to inject. Let `f` be a target function:

- Inject **assume** `I` at entry to `f` *if* `read*(f)` has overlap with `read*(I)`.
- At every point in `f` where a memory location `M` is updated inject **assert** `I` after the update *if* `M in read*(I)`. Also, if `I` is an update invariant, before the update inject a memory snapshot save.

Notice that we do not inject any invariants in functions which are not verification target. However, the set of target functions may need to be extended because of invariants, as described later.

Figure 7: Genericity

```
invariant [I1] global<S<u64>>(0).value > 1;
```

Figure 8: Modular Verification and Invariants

```
module Store {
  struct T has key { x: u64 }
  public fun read(): u64 { borrow_global<S>(0).x }
  public fun write(x: u64) { borrow_global_mut_<S>(0).x = x }
}
module Actor {
  use Store;
  invariant global<S>(0).x > 0;
  public fun set(x: u64) {
    if (x == 0) then abort 1;
    Store::set(x);
  }
}
```

the invariant.

Consider the example in Fig. 8. The module `Store` provides an API for some storage location which is shared between a set of modules. The module `Actor`, one of those modules, establishes an invariant on the content of the store. When `Actor` is verified, one must also verify the function `Store::write`, because this invariant is verification target. (In this example, verification cannot succeed, because the function `Store::write` is not restricting the values for the parameter `x`; we see in the next section how to fix this.)

In general, the set of additional functions to verify is computed as follows. Let `I` be a target invariant which appears in some target module, and `f` some function in the dependency relation. If `modify(f)` has an overlap with `read*(I)` then `f` needs to be added to the target functions. Notice it is not `modify*(f)`; only direct modifications make a function to a verification target (with exceptions as discussed in the next section).

3.4.4 Suspending Invariants

The example in Fig. 8 is not quite right from a design viewpoint, since a global store accessible to everybody is constrained by a specific module. Consequently, it cannot be successfully verified. Fig. 9 modifies the example to fix this. First, Move's **friend** mechanism is used to restrict visibility of `Store::write` to the `Actor` module. Note one could add other modules to the friends list as needed. Second, the `Store::write` function is declared to *suspend invariant evaluation to callers*. Only private and friend functions can have such a declaration, ensuring the all call sites are known and the suspended invariants are actually verified in all call contexts. An invariant needs to be explicitly marked via `[suspendable]` do be eligible for suspension.

When an invariant `I` is suspended for a function `f`, the injection scheme changes as follows:

Figure 9: Suspension of Invariants

```
module Store {
  friend Actor;
  ...
  public(friend) fun write(x: u64) {
    borrow_global_mut_<S>(0).x = x
  }
  spec write { pragma suspend_invariants; }
}
module Actor {
  ...
  invariant [suspendable] global<S>(0).x > 0;
}
```

- At the de nition side of `f`, `I` is neither assumed nor asserted.
- At every call side of `f` (whether opaque or inlined), the invariant is asserted right after the call. It will also be assumed at the entry point of the caller.
- Instead of `f` becoming a target function because it modi es the memory read in `I` (see above paragraph about modular veri cation), all callers will become target functions.
- If the caller is itself suspended, the process is instead continued with the parent callers.

Once a function is suspended, automatically all functions it calls which modify memory e ected by the suspended invariants are suspended as well. This is because when those functions are called, the relevant invariants cannot be assumed to hold, and therefore it is likely not fruitful to try to verify something related to them.

For update invariants it should be noted that suspension may change their meaning, depending on the form of the predicate. Without suspension, an update invariant is implemented by snapshotting the memory before the update and then asserting a predicate after the update which refers to the previous state and the current one. For suspended update invariants, the snapshot is taken *before* the suspended function is called, and the assertion injected *after* it returns, which might be earlier resp. later states. An example of an update invariant which works well for suspension is e.g. a requirement for a monotonically increasing value, as in **invariant** `[suspendable]` **old**`(value()) <= value()`.

Methodologically, the suspension mechanism should be used with care, because it may complicate the veri cation problem by propagating veri cation errors to more complex application contexts. The Move prover supports a further pragma to suspend invariant veri cation which draws a clear boundary to function systems with suspension. With **pragma** `suspend_invariants_in_body` a

function can be marked to suspend invariants only in its body but ensure they hold at caller side. This is conceptually syntactic sugar for introducing a helper function:

```
public fun f(P) { S }
spec f { pragma suspend_invariants_in_body; }
⇝
public fun f(P) { f'(P) }
fun f'(P) { S } spec f' { pragma suspend_invariants; }
```

3.4.5 Invariant Consistency

TODO(wrwg): Describe solution to the below problem via induction

Notice that invariant injection can lead to inconsistencies. Consider the following code fragment:

```
  invariant [I] forall a: address: global<S>(a).value > 0;
  ⇝
  spec assume global<S>(0).value == 0;
// context, e.g. from a requires
  spec assume I;                          // injected
```

We currently do not check whether an invariant is satis able before we assume it, but rather rely on a generic consistency checker for speci cations.

3.5 Monomorphization

Monomorphization is the process of removing all generic types from a Move program by *specializing the program for all relevant type instantiations*. Like with genericity in most modern program languages, this is possible in Move because the number of instantiations is statically known for a given program fragment. For veri cation of Move, monomorphization greatly improves the performance of the backend solvers (see **??**).

3.5.1 Basic Monomorphization

Figure 10: Basic Monomorphization

```
struct S<T> { .. }
fun f<T>(x: T) { g<S<T>>(S(x)) }
fun g<S:key>(s: S) { move_to<S>(.., s) }
⇝
struct T{}
struct S_T{ .. }
fun f_T(x: T) { g_S_T(S_T(x)) }
fun g_S_T(s: S_T) { move_to<S_T>(.., s) }
```

To verify a generic function, monomorphization skolemizes the type parameter into a given type. It then, for all functions which are inlined, inserts their code specializing it for the given type instantiation, including specialization of all used types. Fig. 10 sketches this approach.

The underlying conjecture is that if we verify `f_T`, we have also veri ed it for all possible instantiations. However, this statement is only correct for code which does not depend on runtime type information.

3.5.2 Type Dependent Code

The type of genericity Move provides does not allow for full type erasure as often found in programming languages. That is because types are used to *index* global memory (e.g. **global**`<S<T>>(addr)` where `T` is a generic type). Consider the following Move function:

```
fun f<T>(..) { move_to<S<T>>(s, ..); move_to<S<u64>>(s, ..) }
```

Depending on how `T` is instantiated, this function behaves di erently. Speci cally, if `T` is instantiated with `u64` the function will always abort at the second **move_to**, since the target location is already occupied.

The important property enabling monomorphization in the presence of type dependent code is that one can identify the situation by looking at the memory accessed by code and injected speci cations. From this one can derive *additional instantiations of the function* which need to be veri ed. For the example above, verifying both `f_T` and an instantiation `f_u64` will cover all relevant cases of the function behavior. Notice that this treatment of type dependent code is speci c to the problem of veri cation, and cannot directly be applied to execution.

The algorithm for computing the instances which require veri cation works as follows. Let `f<T1,..,Tn>` be a veri ed target function which has all speci cations injected and inlined function calls expanded.

- Foreach memory `M in modify(f)`, if there is a memory `M' in modify(f)+read(f)` such that `M` and `M'` can unify via `T1,..,Tn`, collect an instaniation of the type parameters `Ti` from the resulting substitution. This instantiation may not assign values to all type parameters, and those unassigned parameters stay as is. For instance, `f<T1, T2>` might have a partial instantiation `f<T1, u8>`.

- Once the set of all those partial instantiations is computed, it is extended by unifying the instantiations against each other. If `<t>` and `<t'>` are in the set, and they unify under the substitution `s`, then `<s(t)>` will also be part of the set. For example, consider `f<T1, T2>` which modi es `M<T1>` and `R<T2>`, as well as accesses `M<u64>` and `R<u8>`. From this the instantiations `<u64, T2>` and `<T1, u8>` are computed, and the additional instantiation `<u64, u8>` will be added to the set.

- If after computing and extending instantiations any type parameters remain, they are skolemized into a given type as described in the previous section.

To understand the correctness of this procedure, consider the following arguments:

- *Direct interaction* Whenever a modi ed memory `M<t>` can in uence the interpretation of `M<t'>`, a uni er must exist for the types `t` and `t'`, and an instantiation will be veri ed which covers the overlap of `t` and `t'`.
- *Indirect interaction* If there is an overlap between two types which in uences whether another overlap is semantically relevant, the combination of both overlaps will be veri ed via the extension step.

Notice that even though it is not common in regular Move code to work with both memory `S<T>` and, say, `S<u64>` in one function, there is a scenario where such code is implicitly created by injection of global invariants. Consider the example in Fig. 7. The invariant `I1` which works on `S<u64>` is injected into the function `g<R>` which works on `S<R>`. When monomorphizing `g`, we need to verify an instance `g_u64` in order to ensure that `I1` holds.

3.6 Translation to Boogie and Z3

3.6.1 Vectors and Extensionality

3.6.2 Encoding

3.6.3 Butterflies

References

[1] Sam Blackshear, Todd Nowacki, Shaz Qadeer, and John Mitchell. The move borrow checker, 2021.

[2] Dave Clarke, Johan Östlund, Ilya Sergey, and Tobias Wrigstad. Ownership types: A survey. In Dave Clarke, James Noble, and Tobias Wrigstad, editors, *Aliasing in Object-Oriented Programming. Types, Analysis and Verification*, volume 7850 of *Lecture Notes in Computer Science*, pages 15{58. Springer, 2013.

[3] The Boogie Team. Boogie Intermediate Veri cation Language.

[4] The CVC Team. CVC5.

[5] The Z3 Team. Z3 Prover.